PeopleOps Multi-Agent Enterprise Platform

Clean Repository & Folder Structure Specification (Backend & Frontend)

Official reference guide describing the directory hierarchy, purpose of every folder, and architectural organization.

Core Architecture Standards & Clean-State Invariants:
- **Single Canonical .env:** Located strictly at project root. Zero duplicate or nested .env files in subfolders.
- **Single Master README.md:** Centrally located at root. Nested template readmes removed.
- **Zero Clutter / Scratch Files:** All one-off database migration and temporary debug scripts removed.
- **Separation of Concerns:** Backend exposes clean REST & SSE streaming APIs on port **8001**; Frontend Vite client runs on port **5173** with automatic reverse-proxy routing via /api.

1. High-Level Repository Layout (Root Directory)

The project root is structured to give any developer an immediate, intuitive view of all high-level subsystems:

Directory / File	Category	Description & Responsibility
backend/	Core Service	FastAPI server, Google ADK multi-agent fleet, NeMo guardrails, Hybrid Graph RAG, PostgreSQL ledger & ChromaDB.
frontend/	Web Client	React 19 + TypeScript + Vite + Tailwind CSS dashboard with SSE streaming AI drawer & portal workflows.
docker/	DevOps	Multi-stage container definitions (Dockerfile.backend and Dockerfile.frontend) for production deployment.
docs/	Documentation	Architecture blueprints, demo scripts, operations guides, threat model, UI screenshots, and PDF guides.
promptfoo/	AI Red-Teaming	Adversarial jailbreak evaluation configs, test matrix datasets, and prompt assertion rules.
scripts/	Operations	Developer tooling and secure public tunneling scripts (start_ngrok.bat, set_authtoken.bat).
.env & .env.example	Configuration	Single canonical environment configuration (database, OpenAI, Gemini, Langfuse, Zoho credentials).
docker-compose.yml	Orchestration	Multi-service Docker compose orchestrator linking backend, frontend, PostgreSQL, and Neo4j.
README.md	Documentation	Comprehensive enterprise documentation, quickstart guide, API contracts, and architecture diagrams.

2. Backend Architecture Structure (backend/)

The backend is built around a domain-driven, modular architecture where multi-agent workflows, data models, compliance engines, and external integrations are cleanly decoupled:

```
backend/
|-- alembic/                # Database migration environment (env.py, versions/)
|-- app/                    # Primary application package
|   |-- agents/             # Google ADK Multi-Agent Fleet (Orchestrator & Specialists)
|   |-- api/v1/             # REST endpoints & SSE streaming routes
|   |-- compliance/         # Regulatory rule engines (HIPAA Safe Harbor 18 & DPDP)
|   |-- core/               # App configuration (loads root .env), DB engine & JWT auth
|   |-- evaluations/        # Automated grid benchmark evaluation matrix (Langfuse + ADK)
|   |-- guardrails/         # NVIDIA NeMo Guardrails boundary (Colang 2.0 rails)
|   |-- memory/             # Multi-turn conversational sliding context & vector recall
|   |-- models/             # Relational SQLAlchemy models (User, Leave, Timesheet, Session)
|   |-- rag/                # Hybrid Knowledge Graph (NetworkX/Neo4j) & vector search
|   |-- schemas/            # Pydantic v2 data contracts & structured action cards
|   |-- services/           # Domain business services & Zoho REST v2 circuit breaker
|   |-- tools/              # Domain-specific Google ADK tools (leave, policy, health, etc.)
|   \-- main.py             # FastAPI entrypoint, CORS & application lifespan
|-- data/                   # ChromaDB vectors, token vault, policy catalog & seed data
|-- reports/                # Benchmark evaluation result JSONs
|-- scripts/                # 7 Production maintenance & utility scripts
|-- tests/                  # Automated pytest suite (E2E memory, security, API routing)
\-- uploads/                # Temporary buffer for KYC document verification
```

Backend Subdirectory Functional Catalog

Subdirectory	Core Responsibility	Key Modules / Files
app/agents/	Multi-Agent fleet powered by Google ADK and LangGraph. Root orchestrator performs dual-rail routing to specialized subagents with resilient fallback.	`orchestrator.py, root_llm_agent.py, leave_agent.py, timesheet_agent.py, policy_agent.py, health_agent.py, statutory_critic.py, prompt_strategies.py`
app/api/v1/	REST API and Server-Sent Events (SSE) streaming endpoints serving user chat, leave ledger, timesheets, and authentication.	`agents.py, auth.py, leaves.py, timesheets.py, policies.py, onboarding.py, prompts.py`
app/compliance/	Regulatory safety and legal engines ensuring 100% adherence to Maharashtra, Karnataka, and Delhi Shops & Establishments Acts.	`phi_engine.py (HIPAA Safe Harbor 18 & DPDP redaction), statutory_rules.py (Labour Law Rules)`
app/core/	Settings and database engine. Securely loads the single root .env and initializes SQLAlchemy async PostgreSQL connection pool.	`config.py, database.py, security.py, id_generator.py, llm_bridge.py`
app/guardrails/	NVIDIA NeMo Guardrails boundary intercepting prompt injections, jailbreaks, and out-of-scope employee queries.	`nemo_boundary.py, guardrails_manager.py, config/ (Colang 2.0 policies)`
app/memory/	Conversational memory manager supporting 5-session sliding context, vector similarity recall, and reboot disk snapshotting.	`conversation_memory.py`
app/rag/	Hybrid Knowledge Graph and Vector RAG engine combining NetworkX, Neo4j Aura Cloud, and local ChromaDB embeddings.	`knowledge_graph.py, agentic_rag.py, graph_rag_service.py, entity_extractor.py, graph_schema.py`
app/services/	Domain business logic and external integrations including Zoho People REST v2 sync and Langfuse dynamic prompt retrieval.	`leave_service.py, timesheet_service.py, zoho_service.py, auth_service.py, langfuse_prompt_service.py`
app/tools/	Domain-specific Google ADK tool implementations for leave balance queries, timesheet logging, policy search, and health certs.	`hr_tools.py, leave/, timesheet/, policy/, health/, zoho/`
data/	Persistent data assets: encrypted token vault, ChromaDB vector store, exported trace logs, and static statutory acts.	`chroma_db/, token_vault/, traces/, company_policy_2026.json, maharashtra_labour_law.json, seed_data.py`
scripts/	7 Production utility scripts for benchmark validation, Langfuse synchronization, graph seeding, and PDF document generation.	`test_full_suite.py, run_golden_dataset_benchmark.py, seed_neo4j_graph.py, sync_db_mirror.py, push_prompt.py, generate_architecture_pdf.py`

3. Frontend Architecture Structure (frontend/)

The frontend is a modern React 19 single-page application built with TypeScript, Vite, and Tailwind CSS. It is structured into presentation components, functional page portals, global contexts, and centralized API utilities:

```
frontend/
|-- public/                # Static web assets, favicon & SVG icon sprite
|-- src/                   # Application source code
|   |-- assets/            # Brand graphics, icons & visual hero assets
|   |-- components/        # Reusable UI component library (AgentChatDrawer, Navbar, etc.)
|   |-- context/           # React Context providers (DemoContext instant role switcher)
|   |-- data/              # Static enterprise feature & architecture deep-dive catalogs
|   |-- pages/             # Portal views (Dashboard, Leave, Timesheet, Policy, Onboarding)
|   |-- utils/             # Formatting helpers for currency, dates, and status badges
|   |-- api.ts             # Central Axios HTTP client & SSE streaming event listener
|   |-- App.tsx            # Root router, navigation shell & modal container
|   |-- index.css          # Tailwind CSS design system tokens & animation styles
|   |-- main.tsx           # React 19 DOM bootstrap entrypoint
|   \-- types.ts           # Strict TypeScript interface contracts matching backend models
|-- package.json           # NPM project manifest, dependencies & build scripts
\-- vite.config.ts         # Vite build bundler & reverse proxy (/api -> http://127.0.0.1:8001)
```

Frontend Subdirectory Functional Catalog

Directory / File	Role & Responsibility	Key Features / Included Modules
src/components/	Reusable UI component library implementing the enterprise design system, drawer drawers, and modals.	`AgentChatDrawer.tsx (real-time SSE streaming assistant), AgentFlowCircuit.tsx (live routing circuit), Navbar.tsx (header, reset & role selector), TokenVaultModal.tsx (PII redaction viewer), ZohoPeopleIntegrationHub.tsx (HRMS sync controls)`
src/pages/	Enterprise workflow portals providing dedicated views for executive metrics, leave ledgers, time tracking, and KYC.	`Dashboard.tsx (fleet telemetry & metrics), LeavePortal.tsx (statutory leave ledger), TimesheetPortal.tsx (hours logging & overtime audit), PolicyPortal.tsx (statutory RAG search), ReportsPortal.tsx (CSV exporter), OnboardingPortal.tsx (DPDP document upload)`
src/context/	Global React Context providers managing authentication tokens, active user personas, and role permissions.	`DemoContext.tsx (allows instant zero-friction switching between Employee, Manager, and Admin personas with full RBAC updates)`
src/data/	Static domain datasets, architectural feature catalogs, and educational walkthrough copy.	`featureDeepDive.ts (comprehensive deep-dive catalog of enterprise security, guardrails, and compliance capabilities)`
src/utils/	Formatting utilities ensuring standardized display of dates, currency figures, and statutory compliance status pills.	`formatters.tsx (formatDate, formatDuration, getStatusBadgeColor, calculateLeaveAccrualSummary)`
src/api.ts	Unified Axios HTTP client configured with baseURL '/api' and real-time Server-Sent Events (SSE) streaming reader.	`Provides typed methods for chat completion, leave applications, timesheet submissions, KYC uploads, and memory resets.`
src/types.ts	TypeScript domain model definitions matching backend Pydantic schemas 1-to-1.	`User, LeaveRequest, TimesheetEntry, AgentChatResponse, StructuredAction, CitationItem, MemoryTurn`
vite.config.ts	Vite development server configuration featuring built-in reverse proxy.	`Transparently proxies all '/api' requests to the FastAPI backend running on http://127.0.0.1:8001.`

4. Operational Quick Reference & Development Commands

Subsystem	Port	Startup Command	Verification / Health Check
Backend API	8001	`uvicorn app.main:app --port 8001`	`http://127.0.0.1:8001/api/v1/health`
Frontend UI	5173	`npm run dev`	`http://localhost:5173 (proxies /api to 8001)`
Test Suite	N/A	`pytest tests/test_deep_end_to_end_memory_and_features.py`	9 / 9 tests passing (100% test pass rate)
Frontend Build	N/A	`npm run build (tsc -b && vite build)`	Zero TypeScript or bundle errors (1.61s)